

TRAFFIC ACCIDENT ANALYSIS

Exploratory Data Analysis on Traffic Accident Dataset

AIM

To perform Exploratory Data Analysis (EDA) on a traffic accident dataset using Python and identify useful patterns based on time, location/area, severity, accident type, and vehicle type through data cleaning, analysis, and visualization.

DATASET

Dataset Description

The Traffic Accident dataset contains accident records with information about the date and time of occurrence, location/area, accident type, severity, and vehicle type.

Dataset Columns

Column	Description
Date	Date on which the accident occurred
Time	Time at which the accident occurred
Location/Area	Area or location of the accident
Accident Type	Type/category of accident
Severity	Severity level of the accident
Vehicle Type	Vehicle involved in the accident

Dataset File

Traffic_Accident_Dataset.csv

OBJECTIVES

1. To understand the structure of the Traffic Accident dataset.
2. To identify missing values and duplicate records.
3. To clean and convert date/time fields.
4. To analyze accidents by time and identify accident trends
5. To identify accident-prone locations/areas.
6. To analyze accident severity distribution.
7. To analyze accidents by vehicle type and accident type.
8. To visualize important accident patterns using graphs.
9. To write key observations from the analysis.

▼ IMPORT LIBRARIES     

```
[2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
print("Libraries imported successfully!")
```

Libraries imported successfully!

▼ LOAD DATA     

```
[5]: df = pd.read_csv("Traffic_Accident_Dataset.csv")
print("Dataset loaded successfully!")
df.head()
```

Dataset loaded successfully!

```
[5]:
```

	Date	Time	Location/Area	Accident Type	Severity	Vehicle Type
0	2025-01-01	06:54:00	Central	Collision	Moderate	Truck
1	2025-01-01	19:15:00	Highway	Collision	Minor	Motorcycle
2	2025-01-01	14:45:00	Central	Hit-and-run	Severe	Motorcycle
3	2025-01-01	10:42:00	Suburban	Collision	Moderate	Auto
4	2025-01-01	07:06:00	Highway	Hit-and-run	Minor	Motorcycle

▼ EXPLORE DATA

```
[6]: print("Dataset Shape:")
      print(df.shape)
      print("\nColumn Names:")
      print(df.columns.tolist())
```

Dataset Shape:

(5200, 6)

Column Names:

['Date', 'Time', 'Location/Area', 'Accident Type', 'Severity', 'Vehicle Type']

```
[7]: print("Data Types:")
      print(df.dtypes)
```

Data Types:

Date str

Time str

Location/Area str

Accident Type str

Severity str

Vehicle Type str

dtype: object

```
[8]: print("Missing Values:")
      print(df.isnull().sum())
```

Missing Values:

Date	0
Time	0
Location/Area	0
Accident Type	0
Severity	0
Vehicle Type	0
dtype: int64	

```
[9]: print("Number of Duplicate Rows:")
      print(df.duplicated().sum())
```

Number of Duplicate Rows:

0

▼ CLEAN DATA



```
[10]: # Convert Date and Time into appropriate formats
      df["Date"] = pd.to_datetime(df["Date"], errors="coerce")
      df["Time"] = pd.to_datetime(df["Time"], format="%H:%M:%S",
      errors="coerce")
      # Remove duplicate rows
      df = df.drop_duplicates()
      print("Data cleaning completed successfully.")
      print("Final dataset shape:", df.shape)
```

Data cleaning completed successfully.

Final dataset shape: (5200, 6)

```
[11]: print("Missing values after cleaning:")
      print(df.isnull().sum())
      print("\nDuplicate rows after cleaning:")
      print(df.duplicated().sum())
```

Missing values after cleaning:

Date	0
Time	0
Location/Area	0
Accident Type	0
Severity	0
Vehicle Type	0

dtype: int64

Duplicate rows after cleaning:

0

ACCIDENT ANALYSIS BY TIME

```
[13]: df["Hour"] = df["Time"].dt.hour
      hourly_accidents = df["Hour"].value_counts().sort_index()
      print("Accidents by Hour:")
      print(hourly_accidents)
      print("\nPeak accident hour:",
            hourly_accidents.idxmax(),
            "with", hourly_accidents.max(), "accidents.")
```

Accidents by Hour:

Hour	
0	219
1	222
2	235

3	225
4	225
5	209
6	186
7	230
8	210
9	194
10	204
11	213
12	227
13	194
14	240
15	182
16	252
17	200
18	222
19	205
20	228
21	241
22	226
23	211

Name: count, dtype: int64

Peak accident hour: 16 with 252 accidents.

```
[14]: df["Month"] = df["Date"].dt.to_period("M").astype(str)
monthly_accidents = df.groupby("Month").size()
print("Monthly Accident Counts:")
print(monthly_accidents)
```

Monthly Accident Counts:

Month

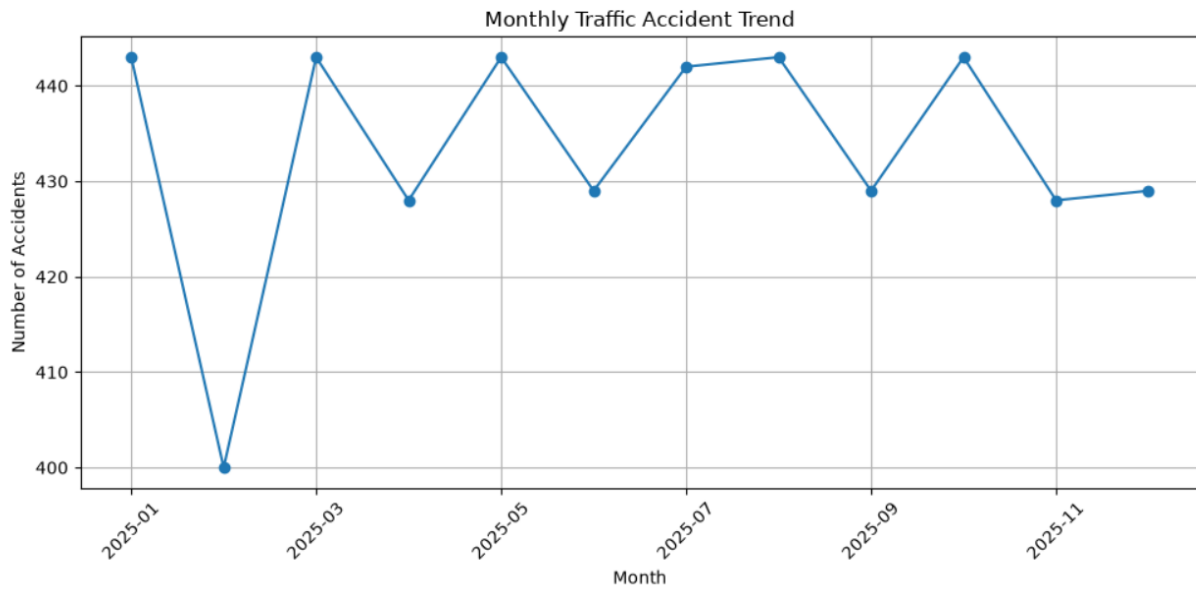
2025-01	443
2025-02	400
2025-03	443
2025-04	428
2025-05	443
2025-06	429
2025-07	442
2025-08	443
2025-09	429
2025-10	443
2025-11	428
2025-12	429

dtype: int64

▼ ACCIDENT TREND LINE CHART

🔄 ⬆ ⬇ ⬇ ⬆ 🗑

```
[15]: plt.figure(figsize=(10, 5))
monthly_accidents.plot(kind="line", marker="o")
plt.title("Monthly Traffic Accident Trend")
plt.xlabel("Month")
plt.ylabel("Number of Accidents")
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```



ACCIDENT ANALYSIS BY LOCATION / AREA



```
[16]: area_accidents = df["Location/Area"].value_counts()
print("Accidents by Area:")
print(area_accidents)
```

Accidents by Area:

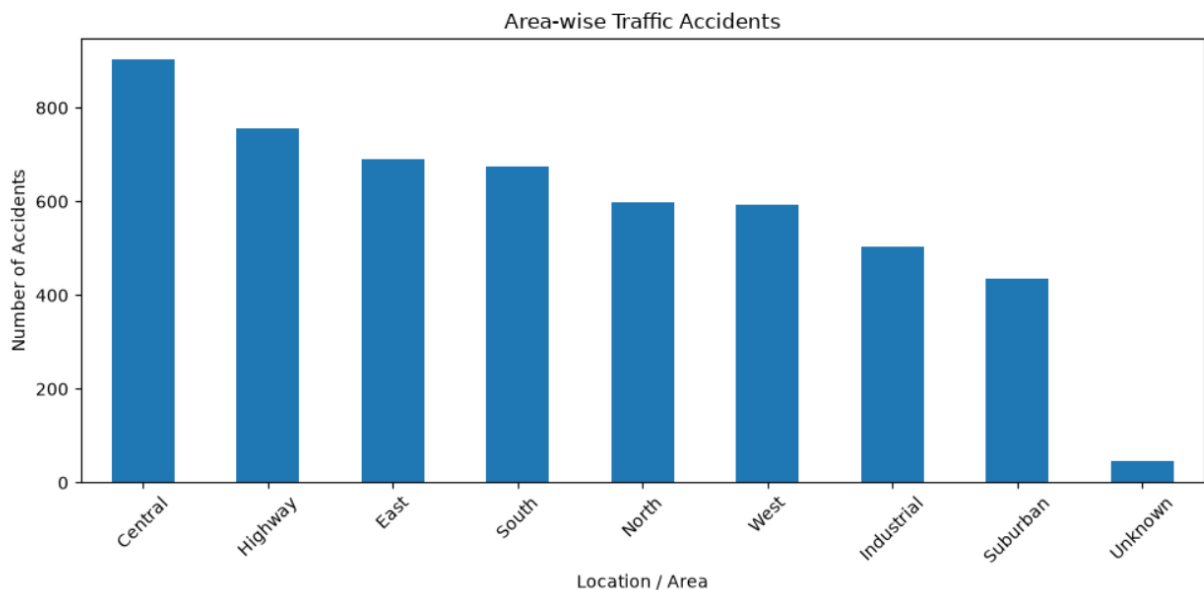
Location/Area

Central	902
Highway	757
East	690
South	674
North	598
West	594
Industrial	504
Suburban	436
Unknown	45

Name: count, dtype: int64

AREA-WISE BAR CHART

```
[17]: plt.figure(figsize=(10, 5))
      area_accidents.plot(kind="bar")
      plt.title("Area-wise Traffic Accidents")
      plt.xlabel("Location / Area")
      plt.ylabel("Number of Accidents")
      plt.xticks(rotation=45)
      plt.tight_layout()
      plt.show()
```



SEVERITY ANALYSIS

```
[18]: severity_counts = df["Severity"].value_counts()
      print("Accidents by Severity:")
      print(severity_counts)
```

Accidents by Severity:

Severity

Minor 2396

Moderate 1690

Severe 827

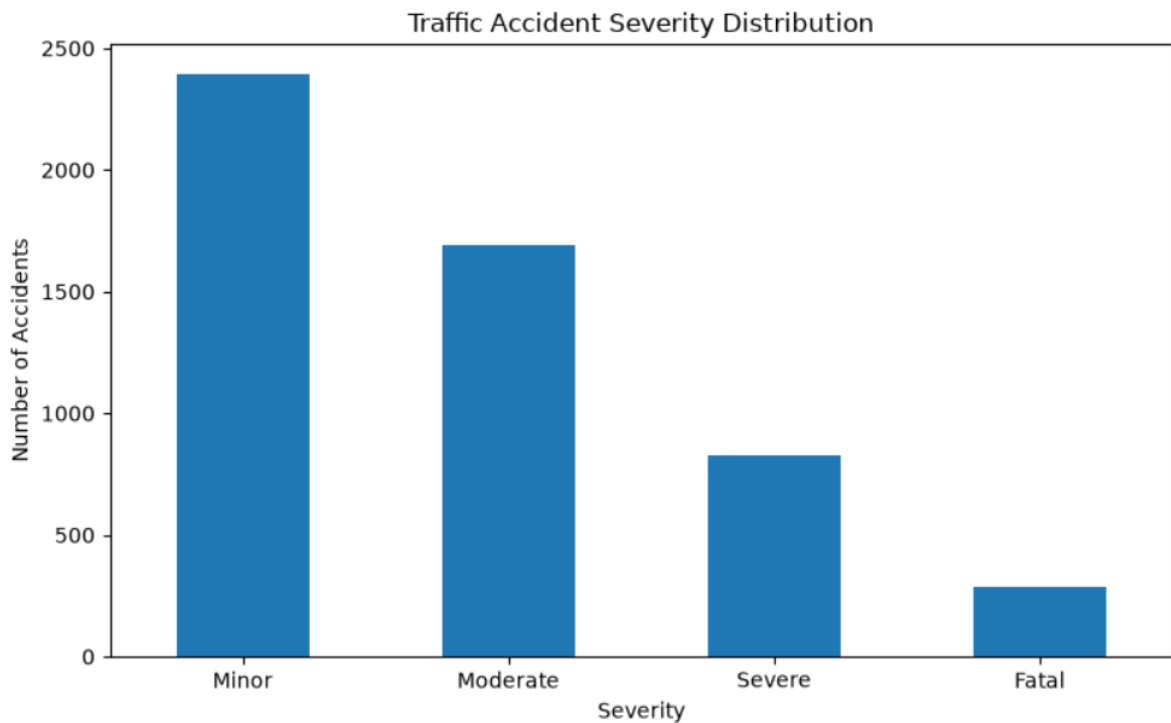
Fatal 287

Name: count, dtype: int64

SEVERITY DISTRIBUTION PLOT



```
[19]: plt.figure(figsize=(8, 5))
severity_counts.plot(kind="bar")
plt.title("Traffic Accident Severity Distribution")
plt.xlabel("Severity")
plt.ylabel("Number of Accidents")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



ACCIDENT ANALYSIS BY VEHICLE TYPE ¶

```
[20]: vehicle_accidents = df["Vehicle Type"].value_counts()
print("Accidents by Vehicle Type:")
print(vehicle_accidents)
```

Accidents by Vehicle Type:

Vehicle Type

Car 2120

Motorcycle 1050

Truck	671
Auto	644
Bus	418
Bicycle	272
Unknown	25

Name: count, dtype: int64

▼ ACCIDENT TYPE ANALYSIS



```
[21]: accident_type_counts = df["Accident Type"].value_counts()
print("Accidents by Accident Type:")
print(accident_type_counts)
```

Accidents by Accident Type:

Accident Type

Collision	1998
Rear-end	1250
Single Vehicle	852
Hit-and-run	569
Pedestrian	531

Name: count, dtype: int64

```
[22]: print("Most common accident type:",
accident_type_counts.idxmax(),
"with", accident_type_counts.max(), "records.")
print("Most common vehicle type:",
vehicle_accidents.idxmax(),
"with", vehicle_accidents.max(), "records.")
print("Most frequent accident area:",
area_accidents.idxmax(),
"with", area_accidents.max(), "records.")
```

Most common accident type: Collision with 1998 records.

Most common vehicle type: Car with 2120 records.

Most frequent accident area: Central with 902 records.

OBSERVATIONS

1. The dataset contains 5,200 traffic accident records accident type, severity, and vehicle type. across six fields: date, time, location/area.
2. The dataset contains no missing values and no duplicate records, so no rows were removed during duplicate handling.
3. The Central area has the highest number of recorded accidents, while Suburban has fewer recorded accidents among the named areas.
4. Minor accidents are the most common severity category, followed by Moderate, Severe, and Fatal accidents.
5. Cars are the most frequently recorded vehicle type, and Collision is the most common accident type in the 5. dataset.

CONCLUSION

The Traffic Accident EDA project demonstrates how Python can be used to inspect, clean, analyze, and visualize accident data. The analysis identifies patterns across time, location, severity, vehicle type, and accident type. The trend line, area-wise bar chart, and severity distribution plot provide a clear visual summary of the major accident patterns in the dataset.

· Python 3 (ipykernel) · 31 cells · 19 outputs